MASooda — Project Report

Nov 10, 2026 · @Abdur Rehman

Multi-Agent System for Hierarchical, Priority-Aware and Resilient Urban Air Traffic Management.

1. Scenario (as built)

MASooda simulates a heterogeneous fleet of up to 500 autonomous drones sharing low-altitude urban airspace over the IIT Hyderabad campus and Hyderabad city roads. It is live at masooda.onrender.com.

- **Fleet:** emergency/medical drones (priority 0), passenger eVTOLs (priority 1) and parcel-delivery drones (priority 2), each with its own mass, rotors, battery pack and cruise speed.
- **Airspace:** corridors follow the road network: 28 traced campus roads (105 junctions, 13 vertiports: one hospital, three depots, nine hubs) inside a geofence, or OpenStreetMap Hyderabad roads. Each corridor is a 20 m tube with one lane per direction. Seven altitude layers sit 15 m apart from 30 m to 120 m. Eastbound traffic uses even layers and westbound uses odd ones. The top layer, L6, is reserved for medical emergencies.
- **Local awareness:** each drone sees only neighbours within 40 m over vehicle-to-vehicle (V2V) links.
- **Two tiers:** drones coordinate locally (car-following, boids, ADMM deconfliction). A Drone Traffic Controller (DTC) runs every 2 s over six sectors × seven layers, each cell with a nominal capacity of 15 drones.
- **Adversity:** drones can broadcast spoofed positions. The DTC link can drop. Batteries can lose capacity, lose a cell or overheat. Weather cells and no-fly zones can appear mid-run.

2. Objective and what was achieved

All five design objectives in the proposal were implemented and run in the same simulation loop.

#	Objective (proposal)	Achieved
1	**Local governance:** merge, keep separation, resolve short-range conflicts from neighbour data only	Car-following ($v \leqslant (g - 12\ m)/1\ s$), boids separation/alignment/cohesion, closest-point-of-approach avoidance, consensus ADMM on conflict groups within 20 m
2	**Global governance:** capacity limits, macro-routing, right of way as low-rate directives, with fallback	DTC every 2 s: sector × layer allocation, capacity moves, layer pricing, reroute/hold/ground-stop/TFR commands; on link loss drones keep the last directive
3	**Priority arbitration** in both local and global decisions	emergency > eVTOL > delivery in DTC allocation (reserved emergency layer) and in ADMM weights (10 / 3 / 1)
4	**Adversarial resilience** against falsified state	Trusted-range check (> 3 m disagreement is corrected) plus W-MSR with f = 1; spoofed reports accepted: 0 defended vs. 193 per run undefended
5	**Formal safety verification** independent of which tier acted	STL $G(d \geqslant 8\ m)$ with live robustness $\rho(t)$, layer-aware distance; optional F(reach destination) spec

Research question. *Can a two-tier architecture give scalable, priority-aware, verifiably safe traffic management that is robust to faulty agents and beats purely centralized or decentralized control?*

On the final road-corridor engine (100 drones, 20 seeds): the hierarchical and decentralized designs both delivered 95.7% of flights with 0 near misses. The centralized baseline, acting on reports refreshed at its update rate, had 9.8 near misses and 1.1 collisions per run and delivered 88.0%. The hierarchical design matched decentralized safety. Its DTC layer showed no measurable extra benefit in nominal traffic.

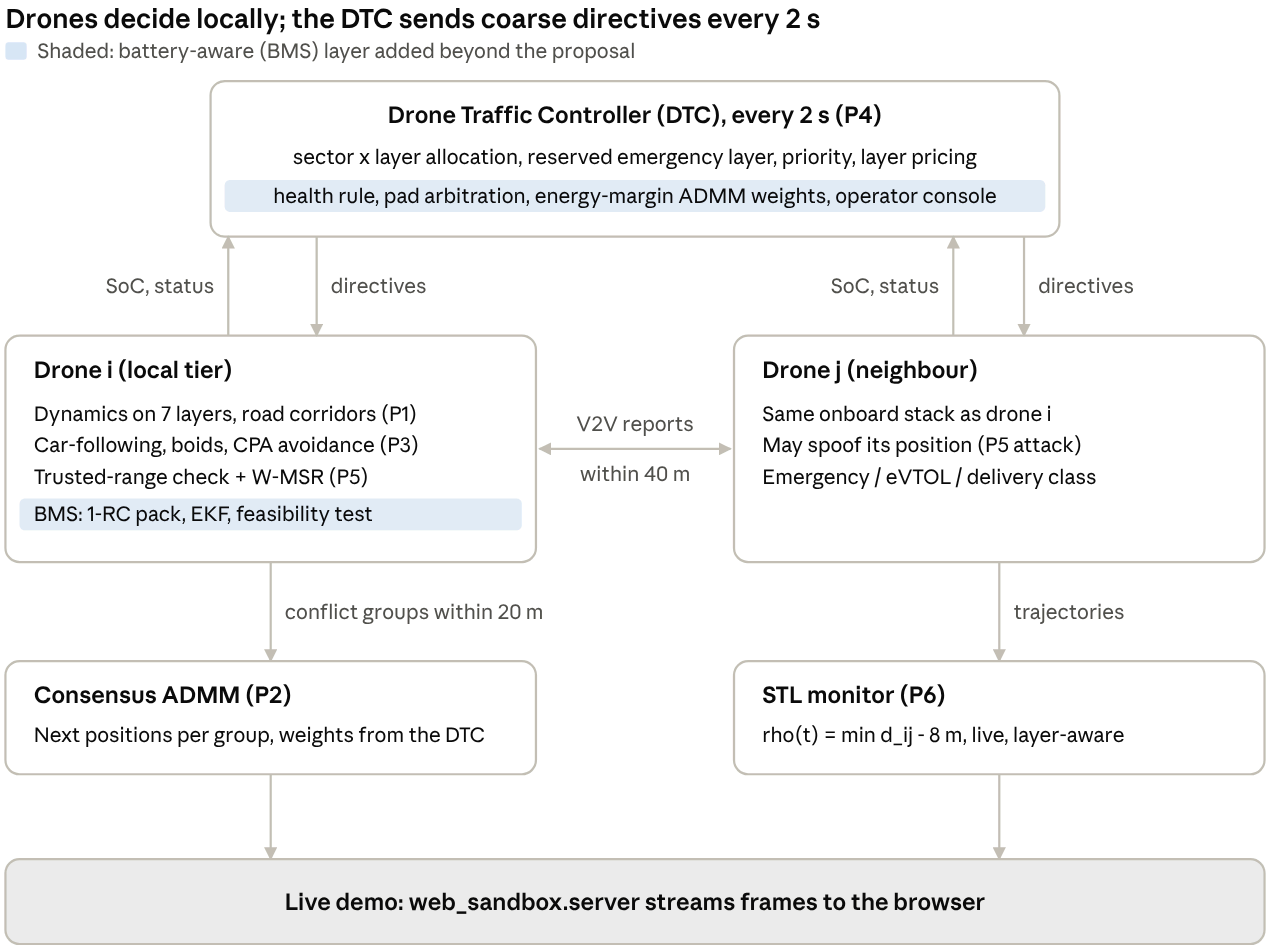
3. Outcomes delivered

All six expected outcomes are delivered and visible in the live demo.

#	Outcome (proposal)	How it is demonstrated now	Code
1	Simulation of N heterogeneous drones self-organising into corridors, with a light overseer	Live 2D map plus 2.5D side view: drones on road corridors over 7 layers, DTC layer moves and reallocation shown on the map; up to 500 drones	`simulation/route_engine.py`, `web_sandbox/`
2	Priority arbitration across drone classes	*IITH medical emergency* preset: a blood-delivery drone gets the reserved layer and right of way; emergency delay and on-time KPIs; delay on lower-priority traffic measured	`simulation/overseer.py`, `route_priority.py`
3	Resilience to adversarial agents	*GPS spoofing* preset: reported vs. estimated position drawn on the map; 0 spoofed reports accepted with the defence vs. 193 per run without	`resilience/`
4	Hierarchical vs. centralized vs. decentralized	Four architectures from identical seeds (initial-conditions hash checked): hierarchical, centralized, decentralized, decentralized-undefended	`simulation/centralized.py`, `route_centralized.py`

#	Outcome (proposal)	How it is demonstrated now	Code
5	Formal safety guarantee (STL)	Live robustness ρ(t) panel that dips at a violation; near-miss and crash counters	`verification/`
6	Graceful degradation when the overseer link drops	*DTC outage* preset (80–150 s): drones keep their last directive, commands queue, no near miss	`simulation/overseer_link.py`

System architecture as built



MASooda architecture as built · two tiers, BMS layer, live demo

Each drone runs its onboard stack every tick and talks to neighbours over V2V. The DTC acts every 2 s on coarse state, including each drone's estimated battery state. ADMM and the STL monitor sit below both tiers, and their outputs stream to the live demo.

Mapping to our coursework

Requirement	Course modules	Where it is in the code
Local rules / neighbourhood behaviour	Graph Theory Basics, Consensus & Topology Conditions	`simulation/comm_graph.py` (radius/kNN graph), `local_behavior/boids.py`
Centralized overseer + local tier interplay	Consensus Optimization, Distributed Learning & Tracking	`conflict_resolution/` (consensus ADMM), `simulation/overseer.py`, `simulation/layer_agreement.py`
Corridor merging	Formation Control, Vehicle Platooning	`local_behavior/`, `simulation/route_formation.py` (car-following, platoon stats)
Priority arbitration	Epistemic Logic, Behavioural Planning with EL	`simulation/overseer.py` (priority-aware allocation), `simulation/route_priority.py`
Adversary handling	Malicious Attacks & Resilient Consensus	`resilience/wmsr.py`, `defense.py`, `attack.py`
Formal safety guarantees	STL Basics, STL Robustness, Mission Planning with STL	`verification/stl_monitor.py`, `live_monitor.py`
Complex arbitration + safety together	Complex Mission Planning with STL & EL	BMS feasibility test and pad arbitration in `simulation/drone_health.py` (energy constraints feeding priority)

Final team structure

Each role delivered its module. The "Delivered by" column follows git authorship. Work beyond the proposal is a seventh line of work, all by Lalith.

#	Role	Primary syllabus topic	Core deliverable	Delivered by (git)	Commits
P1	Simulation Engine Lead	Graph Theory + integration	Agent dynamics, comm graph, sim loop, renderer, centralized baseline	Dhadheechi (dynamics, schema, renderer, Week 1); Abdur (engine loop, comm graph, centralized, telemetry)	1 + 11
P2	Strategy & Conflict-Resolution Lead	Consensus Optimization, ADMM	Distributed ADMM deconfliction	Lalith; the Week 1–4 P2 files were committed inside Sampad's integration commits	57 (all Lalith work)
P3	Local Behaviour & Formation Lead	Formation Control, Platooning	Boids rules, corridor merge, formation metrics	Bhuvan	4
P4	Global Overseer & Priority Lead	Consensus & Topology, Epistemic Logic	Overseer, priority allocation, link failure	Tanmay	5
P5	Adversarial Resilience Lead	Malicious Attacks & Resilient Consensus	Threat model, attack injection, W-MSR, undefended baseline	Sampad (also weekly integration merges)	28
P6	Formal Verification & Demo Lead	STL Basics, STL Robustness	STL monitor, metrics, Week 6 dashboard and plots	Abdur (Week 1 STL monitor committed by Sampad)	in P1 count
—	Extensions beyond scope	Battery management, estimation	Road-corridor simulator, maps, BMS, live demo, paper	Lalith	in P2 count

Week-by-week roadmap as executed

Weeks 1–4 were committed and merged on 1 November, Week 5 on 5–6 November and Week 6 on 7 November. Lalith's extensions ran from 6 to 10 November.

Week	P1 Sim	P2 ADMM	P3 Formation	P4 Overseer	P5 Resilience	P6 STL / Demo
1	Dynamics, schema, renderer (Dhadheechi)	Two-agent ADMM QP solver	Boids formulation (PDF + prototype)	Overseer contract	Threat model (PDF), attack stub	STL spec draft, toy trajectories
2	Comm graph, engine loop (Abdur)	ADMM adapter, config keys, iteration sweep	Real boids on the graph	Overseer skeleton + tests	Standard consensus baseline	Robustness formulas
3	Integration, live sim run	N-agent ADMM, conflict detection	Corridor selection, merge-gap control	Priority-aware allocation	W-MSR vs. baseline	Live monitor on sim trajectories
4	MVP run (Abdur)	Profiling; 2.5D ADMM demo, `final_demo.py`, SQLite outputs (Lalith)	Cache reset, merge check with ADMM	Overseer link resilience	Falsified-broadcast attack	STL events, near-collision check
4.5	2.5D altitude layers (Sampad)	Layer-aware ADMM	—	Layer allocation	—	—
5	Centralized baseline (Abdur)	E1 ADMM scaling + profiler (Lalith)	E2 formation metrics	E3 priority, E6 degradation (committed by Sampad)	E4 resilience, undefended baseline	Standard metrics (Abdur)
6	Telemetry, harness	ADMM fixes: keep conflict groups whole, fallback on non-convergence (Lalith)	E2 plots	E3/E6 plots	E4 plots	Dashboard, plots, `run_all`, reproducibility (git: Abdur)

Week	P1 Sim	P2 ADMM	P3 Formation	P4 Overseer	P5 Resilience	P6 STL / Demo
7–8 (buffer)	Road-corridor engine, IITH + Hyderabad maps, live demo phases 0–5, deployment, BMS, layer agreement, paper (all Lalith)					

Individual contributions: what each person did

Each line is backed by that person's commits. Git identities: `rlv-iith` and `R Lalith Vishnu` = Lalith; `MohammedAbdurRehman11015` = Abdur; `Sampadram` = Sampad.

- **Lalith (P2, 57 commits):** "I designed and implemented the consensus-ADMM conflict resolver (adapter, conflict groups, scaling experiment E1, profiler, convergence fallback). I then built the road-corridor simulator the final demo and paper run on, traced the IITH campus map, built and deployed the live demo, and added the BMS layer, layer agreement and the paper."
- **Abdur (P1 + P6, 11 commits):** "I built the integrated simulation loop, the communication graph, the centralized baseline and the live STL monitor and metrics. I also committed the Week 6 experiment harness, dashboard and plots."
- **Dhadheechi (P1 + P6, 1 commit):** "I built the Week 1 simulation engine: agent dynamics, the shared schema, mock data, project setup and the top-down renderer."
- **Sampad (P5, 28 commits):** "I designed the threat model and implemented attack injection, the consensus baseline, W-MSR and the report defence, and the decentralized-undefended baseline with experiments E4, E3 (priority) and E6 (degradation). I added 2.5D altitude layers to the grid engine and merged every role's weekly branch."
- **Bhuvan (P3, 4 commits):** "I implemented boids alignment, cohesion and separation, corridor selection and merge-gap control, the ADMM merge check and the E2 formation metrics."
- **Tanmay (P4, 5 commits):** "I built the overseer contract and skeleton, priority-aware corridor allocation, and overseer link-failure handling with graceful degradation."

Note: the P2 ADMM files for Weeks 1–4 (`solver.py`, `constraints.py`, `convergence.py`, `adapter.py`, `detection.py` and their tests and sweeps) reached git through Sampad's integration commits `c75bb8f`, `17d41a0`, `8fc3e47` and `e24249b`. These files belong to the P2 role.

Extra work beyond the proposal (Lalith)

The final demo, the battery-aware layer and the paper were not in the proposal. Lalith built all of them between 6 and 10 November: about 12,400 lines of code now in the tree under his two git identities.

Work	What it adds	Main files	Commits
Road-corridor simulator	Replaces the three straight corridors with road-aligned 20 m tubes, geofence, 7 layers, crash detection, spatial grid for 500 drones; runs all team modules (dynamics, boids, ADMM, overseer, W-MSR, STL)	`simulation/route_engine.py` (2,169 lines), `road_network.py`, `spatial_grid.py`	`10f1e2a`, `23e051b`, `1b6f1b4`
Maps	IIITH campus traced from the campus map (28 roads, 105 junctions, 13 vertiports); Hyderabad OSM roads	`data/iith_campus.geojson`, `scripts/build_iith_campus_map.py`	`a5540d7`, `87b773b`

Work	What it adds	Main files	Commits
Drone health and BMS	Momentum-theory power model; 1-RC Thevenin pack with per-cell spread and thermal model; EKF on (SoC, V_RC, 1/Q); chance-constrained usable energy (z = 1.645); voltage-sag feasibility test; scheduled faults (capacity loss, cell failure, overheating, motor, GNSS)	`simulation/drone_health.py`	`4d330fe`, `31a5d2b`, `a2bee9a`, `fccf49a`
Health-aware traffic management	Endurance-ordered landing-pad arbitration with hold-short; energy-margin ADMM weights with bounds proven in the paper	`drone_health.py`, `route_engine.py`, `conflict_resolution/adapter.py`	`31a5d2b`, `6613869`, `b4f9e55`
Layer agreement	V2V layer negotiation plus DTC congestion pricing that spreads traffic over all layers	`simulation/layer_agreement.py`	`2af5762`

Work	What it adds	Main files	Commits
DTC operator console and weather	Reroute, hold, return to base, ground stop, TFR rectangles, quarantine, battery swap (each checked against remaining energy); wind and storm cells	`simulation/dtc_console.py`	`0961f1c`, `56aafbe`, `f69ef0b`
Scenario presets	10 scripted scenarios with side-by-side variants	`simulation/scenario_presets.py`, `docs/SCENARIO_PRESETS.md`	`775ff5e`
Live demo and deployment	Streaming HTTP server, compact frames with adaptive stride, multi-visitor safety, run export, light theme, section navigation; deployed on Render	`web_sandbox/` (about 3,200 lines)	`632468e`, `5341d26`, `649bba7`, `3905457`
Project audit and integration plan	Audit of the whole repo against `workflow.pdf`, P2 action plan, research-gap note on health-aware UTM	`docs/COMPREHENSIVE_PROJECT_AUDIT.md`, `INTEGRATION_PLAN.md`	`41a43af`, `93f59d3`

Work	What it adds	Main files	Commits
IEEE-style paper	*BMS-in-the-Loop Traffic Management for Urban Drone Fleets* (11 pages); 460 paired-seed runs, second-platform check	`paper/`, `scripts/paper_experiments.py`	`fccf49a` (+ untracked `paper/`)

Promised vs. delivered

The project met every public commitment. It also reached two of the stretch items the proposal warned against promising.

Item	Proposal status	Delivered
2D / 2.5D simulation	Promised	2D map plus 2.5D side view, 7 altitude layers
50–80 agents	Promised	Up to 500 in the live demo; 50–200 measured in the paper
2–3 corridors	Promised	Full road networks (IITH campus, Hyderabad)
One local rule set + one overseer with priority	Promised	Boids + car-following; DTC with reserved emergency layer
One resilient defence vs. one attack type	Promised	Trusted-range check + W-MSR vs. position spoofing
One live STL spec + comparison chart	Promised	Live ρ(t) for separation; four-architecture comparison
Second STL spec (reachability)	Headroom item	Implemented: F(reach destination)
Corridor congestion pricing	Optional extension	Implemented in layer agreement

Item	Proposal status	Delivered
500+ agents	Stretch	Reached: 500 drones at 3.4× real time on 2 vCPUs
Multiple simultaneous attack types	Stretch	Partly: spoofing plus a non-cooperative intruder preset
True 3D rendering	Stretch	Not done (2.5D side view only)
Full epistemic-logic belief modelling	Stretch	Not done
Learned (RL) policies	Stretch	Not done

The demo: what the room sees

The final demo is the MASooda Live Airspace site, run locally with `python -m web_sandbox.server` (port 8765) or opened at masooda.onrender.com. It realises the proposal's split screen: a live simulation on the left and live metrics on the right.

Simulation panel (left). A top-down map of the IITH campus or Hyderabad shows drones coloured by type on road corridors, with vertiports and the geofence. Spoofers show their reported and estimated positions. Below the map is a 2.5D side view of the 7 layers. DTC directives, TFR zones and storm cells are drawn as overlays.

Dashboard (right). It shows STL robustness ρ(t), near-miss and crash counters, emergency delay and on-time rate, and the ADMM timer (iterations, solve time, p95, fallbacks). There is also a V2V network panel, a battery matrix of the whole fleet with per-drone BMS state and cell voltages, and the DTC operator console.

Scenario presets (10).

1. IITH medical emergency: blood delivery in the lunch rush (240 s, 120 drones)
2. Peak-hour vertiport congestion: ground-delay program (300 s, 200 drones)
3. Pop-up restricted area: VIP visit at the Academic Area (240 s, 120 drones)
4. GPS spoofing near the restricted zone (240 s, 100 drones)
5. DTC outage: lost-link procedures, outage 80–150 s (240 s, 100 drones)
6. Weather cell: moving no-fly polygon, wind 8 m/s (300 s, 120 drones)
7. Non-cooperative intruder (180 s, 80 drones)
8. Battery failure mid-route (240 s, 100 drones)

9. Hyderabad heat wave, 44 °C vs. a normal day (300 s, 150 drones)
10. Long-duration operations: 60 minutes with recharging (3600 s, 60 drones)

Demo script (proposal beats → presets). Normal operation → any preset's opening 60 s. Malicious drone plus link drop → presets 4 and 5. Emergency drone → preset 1. Comparison chart → the architecture comparison run from identical seeds. New closing beat → preset 8: a cell fails, the BMS goes CRITICAL, the DTC reserves a pad and other arrivals hold short.

Results

The evaluation ran twice. Week 6 ran E1–E6 on the three-corridor grid engine (141 runs). The paper ran 460 paired-seed runs on the road-corridor engine. The paper is the stronger evidence: the grid engine measured separation horizontally across layers, so its violation counts are inflated.

Week 6 experiments (grid engine, seeds 42–44, `outputs/week6/RESULTS.md`, marked provisional)

Exp.	Owner	Question	Result
E1	P2	ADMM scaling with N	Converged 100% at every N (limit 50); about 10 iterations per solve; solve ≈ 1 ms; ADMM time per tick 0.77 ms at N = 30 to 1.79 ms at N = 75. Limit 20 already converges 99.7%
E2	P3	Merge success and throughput	Throughput rises 0.16 → 0.42 agents/s from N = 30 to 75; merge success near 0; corridors exceed capacity (peak 1.8×) at N = 75
E3	P4	Priority compliance	Compliance 1.0 at N = 30, near 0 from N = 45 (assigned corridor over 15 agents); emergency saves 10–36 s vs. a normal agent; delay on others within ±2 s
E4	P5	Resilience under attack	No measurable attack effect in any architecture (95% CI includes 0); undefended violations 0 → 76 ticks with the attack

Exp.	Owner	Question	Result
E5	P6	STL robustness	Violating fraction: hierarchical 0.46–0.99, centralized 0.03–0.10, decentralized 0 (inflated by the horizontal separation metric)
E6	P4	Graceful degradation	Link down 90–120 s: directive age rose to 31.9 s, violations and throughput unchanged (score 1.0)

Paper evaluation (road-corridor engine, 100 drones, 20 paired seeds)

Measure	Result
EKF SoC error	RMSE 0.21–0.33%, max under 1.3% in all fault conditions
Energy bound	Never over-estimated in nominal flight; after a sudden 30% capacity loss, over-estimated in 9% of samples for about 2 minutes
Low-charge fleet	Chance constraint kept 6.4 more drones grounded and avoided 3.5 in-flight diversions per run; deliveries unchanged (−0.2)
Architectures (nominal)	Hierarchical and decentralized: 95.7% arrived, 0 near misses, min separation 12.5 m. Centralized: 88.0%, 9.8 near misses, 1.1 collisions
Spoofing attack	0 spoofed reports accepted defended vs. 193 undefended; separation unchanged
DTC outage 80–150 s	35 directives lost, no change in outcomes
Scaling 50 → 200 drones	ADMM p95 solve near 0.45 ms; real-time factor 22.2× → 5.7× on one core; arrivals 95.8% → 90.9%
Pad arbitration, ADMM weights	No measurable gain in safety, deliveries or emergency delay in these scenarios; one forced landing (seed 13) from hold-short placement

Testing, reproducibility and limitations

The repository has 421 test functions in 80 files. Runs are deterministic for a given seed.

- **Tests:** unit tests per module (ADMM solver, boids, overseer, W-MSR, STL), integration tests for the route engine, an energy-balance check to 10^{-9}, estimator calibration, pad arbitration, KPI definitions, streaming and replay determinism.
- **Reproducibility:** one config file (`configs/base_config.yaml`); `scripts/run_all.py` regenerates Week 6; an initial-conditions hash proves architectures start identically. The paper's 53 runs repeated on a second platform (Python 3.10 vs. 3.13) gave identical discrete outcomes.
- **Limitations:**
  - The battery, power and thermal models are not calibrated against real flight data.
  - Health telemetry is trusted, so a drone that falsely reports CRITICAL would get right of way.
  - ADMM is a mitigation layer with a convexified per-tick constraint, not a formal guarantee.
  - The threshold thermal response is too slow at 44 °C.
  - Wind changes power use but does not push drones off course.
  - Week 6 grid-engine results are provisional.
- **Future work:** calibrate against public delivery-drone flight data; add predictive thermal and energy STL specs such as G(T_p ⩽ T_max); protect health reports the way W-MSR protects positions; test scenarios where landing capacity is scarce.

Repository statistics (git)

The repository has 106 commits across all branches (15 Sep to 10 Nov 2026). The figures below are from `git shortlog` and from `git blame` on 193 tracked code files (Python, JS, HTML, CSS outside `docs/` and `outputs/`), on branch `bms/publishable` at `22c5400`.

Member	Commits	Lines of code in the tree	Main areas
Lalith	57	12,400	`simulation/route_*`, `drone_health.py`, `layer_agreement.py`, `web_sandbox/`, ADMM profiler and fixes
Abdur	11	12,023 (7,564 from the Week 6 commit)	`simulation/engine.py`, `centralized.py`, `verification/`, `visualization/`, `experiments/`
Sampad	28	4,986 (about 1,660 in P2 files)	`resilience/`, integration merges, P2 Week 1–4 files, 2.5D layers
Bhuvan	4	2,374	`local_behavior/`, P3 scripts and tests
Dhadheechi	1	1,102	`dynamics.py`, `schema.py`, `renderer.py`, `mock_data.py`
Tanmay	5	1,050	`overseer.py`, `overseer_link.py`, P4 tests

The final demo (`web_sandbox/`) and the road-corridor engine behind it are entirely Lalith's commits. Both import the team's modules, so every role's work runs in the final demo.